

Rapport de projet

*Réalisation d'une IA pour le jeu
Breakthrough*

Intelligence Artificielle - L2 Informatique

Université de Lorraine – 2025-2026

Jules Tiburce Hinson

Introduction

Ce rapport présente la réalisation du projet de TP d'intelligence artificielle : programmer le jeu Breakthrough et y intégrer une IA. J'ai construit le projet par étapes - représentation du jeu, validation des coups, génération des successeurs, Minimax, Alpha-Beta, fonctions d'évaluation et Monte-Carlo, en testant chaque composant séparément avant de passer à la suite.

1. Présentation générale du projet

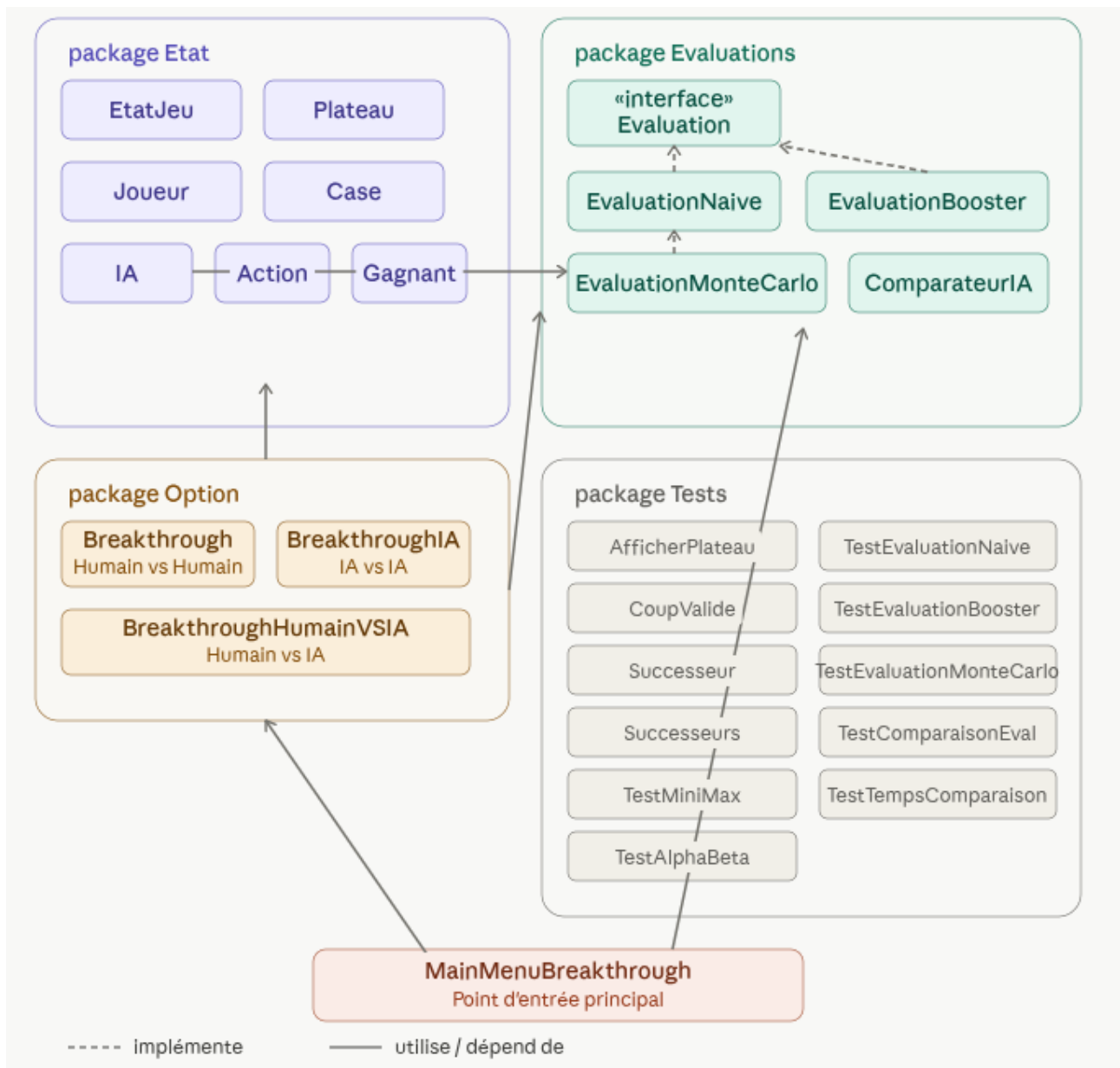
Breakthrough est un jeu à deux joueurs sur un plateau 8×8. Le joueur blanc avance vers les lignes hautes, le joueur noir vers les lignes basses. Un pion peut avancer tout droit si la case est vide, ou en diagonale pour capturer un pion adverse. L'objectif est d'atteindre l'autre côté du plateau avant l'adversaire.

Le projet final propose trois modes de jeu (Humain/Humain, Humain/IA, IA/IA) et trois fonctions d'évaluation : naïve, Booster et Monte-Carlo. Un menu principal permet de configurer la profondeur de recherche et le type d'évaluation.

2. Organisation du projet

Le projet est structuré en quatre packages et une classe principale :

Etat	<i>Logique du jeu : EtatJeu, Plateau, Case, Joueur, Action, Gagnant, IA (Minimax + AlphaBeta)</i>
Evaluations	<i>Interface Evaluation + implémentations : EvaluationNaive, EvaluationBooster, EvaluationMonteCarlo, CompareurIA</i>
Option	<i>Modes de jeu en console : Breakthrough, BreakthroughIA, BreakthroughHumainVSIA</i>
Tests	<i>11 classes de tests : AfficherPlateau, CoupValide, Successeur, Successeurs, TestMiniMax, TestAlphaBeta, TestEvaluationNaive, TestEvaluationBooster, TestEvaluationMonteCarlo, TestComparaisonEval, TestTempsComparaison</i>
Main	<i>MainMenuBreakthrough : point d'entrée, menu interactif en console</i>



3. Modélisation de l'état du jeu

Un état contient le plateau et le joueur qui doit jouer. Chaque position possible est un état au sens du cours, et chaque action valide permet de passer d'un état à un autre. Les pions blancs sont notés B, les noirs N.

Test : AfficherPlateau

J'ai vérifié que le constructeur `EtatJeu(8)` créait bien le plateau initial avec les pions aux bonnes positions. Ce test est fondamental : un état initial incorrect invalide toutes les étapes suivantes.

4. Représentation et validation des actions

Une action est un triplet (ligne, colonne, direction). La direction vaut -1, 0 ou 1 pour les déplacements diagonaux et tout droit. La méthode `actionValide` vérifie : la direction, que la case de

départ contient bien un pion du joueur, et que le déplacement respecte les règles (tout droit si la case est vide, diagonal uniquement pour capturer).

Test : CoupValide

J'ai testé l'action (1, 0, 0) pour le joueur blanc depuis l'état initial - résultat attendu : **true**. J'ai aussi testé un déplacement diagonal hors du plateau - résultat attendu : **false**.

5. Calcul d'un successeur

La méthode successeur crée une copie de l'état courant, déplace le pion, puis change le joueur. Travailler sur une copie est essentiel : l'IA doit explorer plusieurs coups sans modifier la position de départ.

Test : Successeur

J'ai affiché le plateau avant et après un coup. J'ai vérifié qu'un seul pion avait bougé, que l'ancienne case était vide, et que le tour avait changé. Ce test a évité une erreur classique : modifier l'état courant au lieu d'en créer un nouveau.

6. Génération de tous les successeurs

La méthode successeurs parcourt le plateau, teste les trois directions pour chaque pion du joueur courant, et retourne la liste de tous les états accessibles.

Bug corrigé

Au départ, la liste ne contenait que 7 successeurs au lieu de 8 depuis l'état initial. Le problème venait d'une confusion entre déplacement vertical (dépend du joueur) et déplacement horizontal (dépend de la direction). Correction :

```
int ligneArrivee = ligneDepart + directionVerticale;  
int colonneArrivee = colonneDepart + direction;
```

Test : Successeurs

Depuis l'état initial, le joueur blanc doit avoir exactement 8 successeurs (un par pion). Après correction, la liste avait bien 8 éléments. J'ai aussi affiché chaque successeur pour vérifier visuellement qu'un seul pion bougeait à chaque fois.

7. Jeu humain contre humain

J'ai mis en place un mode de jeu en console où l'utilisateur saisit (ligne, colonne, direction). Le programme valide l'action et refuse les coups invalides avec un message explicite. Ce mode a permis de valider manuellement les règles avant d'implémenter l'IA.

Tests : Breakthrough (Package Option)

J'ai testé des coups valides, puis volontairement des coups invalides : hors plateau, diagonal sans capture, case vide, pion adverse. Tous ont été correctement refusés.

8. Fonction d'évaluation naïve (eval0), temps et IA

L'évaluation naïve calcule `nbPionsJoueur - nbPionsAdversaire`. Simple et rapide, elle sert de base pour Minimax. Sa limite : elle ignore complètement la position des pions. Un pion à un coup de la victoire vaut autant qu'un pion au départ.

Test : TestEvaluationNaive

Depuis l'état initial, le score est 0 pour les deux joueurs (même nombre de pions). Après une capture simulée, le joueur avec plus de pions obtient un score positif. Résultats conformes.

Tests : BreakthroughHumainVSIA (Package Option)

L'évaluation naïve a aussi été testée dans une partie humain contre IA. Dans ce mode, l'humain joue contre une IA utilisant Minimax ou Alpha-Beta avec cette fonction d'évaluation.

Le test a permis de vérifier que l'IA choisissait bien un coup valide à chaque tour. Cependant, son comportement reste assez simple. L'IA cherche surtout à éviter de perdre des pions ou à capturer ceux de l'adversaire. Elle ne cherche pas forcément à avancer rapidement vers la ligne d'arrivée, car l'évaluation naïve ne valorise pas la progression.

Tests : BreakthroughIA (Package Option)

Un autre test a été réalisé en mode IA contre IA. Ce test est utile parce qu'il permet d'observer le comportement de l'évaluation naïve sur une partie complète, sans intervention humaine.

Dans ce mode, les deux IA jouent automatiquement. Lorsque l'évaluation naïve est utilisée, les décisions sont cohérentes mais peu stratégiques. L'IA privilégie surtout les positions où elle garde un bon nombre de pions. Elle peut donc parfois rater des coups importants liés à l'avancement.

Mesure du temps de calcul:

Le calcul du temps d'exécution a été ajouté dans toutes les parties où l'IA prend une décision. L'objectif est de mesurer précisément le coût des algorithmes utilisés, en particulier Minimax et Alpha-Beta, lorsqu'ils évaluent un coup.

Pour cela, le temps est mesuré avant et après l'appel à l'algorithme de recherche. La différence entre ces deux instants permet d'obtenir la durée de calcul nécessaire pour déterminer le meilleur coup.

Le temps est exprimé sous deux formats :

- en nanosecondes, pour une mesure très précise,
- en millisecondes, pour une lecture plus compréhensible.

Cette mesure est centralisée dans une méthode spécifique située dans la classe principale. Cette méthode a un double rôle : elle affiche le temps de calcul et renvoie l'état du jeu correspondant au meilleur coup trouvé par l'IA.

Ces mesures ont permis de comparer concrètement les performances des différentes approches. Elles montrent notamment que le temps de calcul augmente rapidement avec la profondeur de recherche, et que l'algorithme Alpha-Beta est plus efficace que Minimax en réduisant le nombre de positions explorées.

Ainsi, l'ajout de cette mesure a permis non seulement de valider le fonctionnement des algorithmes, mais aussi d'analyser leur coût réel en situation d'exécution.

9. Algorithme Minimax

Minimax explore l'arbre des coups jusqu'à une profondeur limite. L'IA maximise son score, l'adversaire le minimise, on suppose que les deux jouent de façon optimale. En profondeur 0, l'algorithme appelle `eval0` pour noter la position.

Test : TestMiniMax

J'ai lancé Minimax avec une profondeur de 1 depuis l'état initial : le programme choisit un coup valide sans planter. En augmentant progressivement la profondeur (1, 2, 3), j'ai observé que le temps de calcul augmente fortement, ce qui est cohérent avec la complexité.

-Complexité Minimax : $O(b^d)$

10. Élagage Alpha-Beta

Alpha-Beta produit le même résultat que Minimax mais évite d'explorer les branches inutiles grâce à deux bornes α et β . Quand $\alpha \geq \beta$, l'exploration de la branche s'arrête. Le gain en temps devient significatif à partir de la profondeur 3.

Test : TestTempsComparaison

J'ai mesuré le temps de calcul avec `System.nanoTime` pour les deux algorithmes. À profondeur 1, la différence est négligeable. À profondeur 3 et au-delà, Alpha-Beta est nettement plus rapide, ce qui confirme l'intérêt de l'élagage.

-Alpha-Beta : $\approx O(b^{(d/2)})$ dans le meilleur cas

11. Évaluation Booster

L'évaluation Booster améliore la naïve en ajoutant un bonus d'avancement. Chaque pion vaut 10 points de base, plus un bonus proportionnel à sa progression sur le plateau : les lignes hautes pour Blanc, les lignes basses pour Noir.

```
if (j == Joueur.joueurBlanc) { score += 10 + i; }
else { score += 10 + (7 - i); }
```

L'idée vient du jeu lui-même : dans Breakthrough, l'objectif est d'avancer. Un pion proche de la ligne adverse est bien plus dangereux qu'un pion au départ. La naïve l'ignore complètement.

Test : TestEvaluationBooster

J'ai fait jouer une IA Booster contre une IA Naïve en IA/IA. Booster jouait mieux et avançait plus vite. Ce test a motivé la comparaison systématique avec `CompareurIA`.

12. Méthode Monte-Carlo

Monte-Carlo estime la probabilité de victoire depuis un état en simulant plusieurs parties aléatoires. La formule du cours est appliquée directement :

```
p = victoires / nombreSimulations
valeur = 2 * p - 1 // dans [-1, +1], multiplié par 100 pour Minimax
```

Bug critique : copie() ne copiait pas tourBlanc

Le bug le plus important du projet se trouvait dans la méthode copie() de EtatJeu. Le champ tourBlanc n'était pas copié : le constructeur EtatJeu(largeur) initialise toujours tourBlanc = true. Résultat : toutes les simulations repartaient avec le joueur Blanc, même quand c'était au tour du Noir — faussant entièrement les estimations de probabilité.

Correction : une seule ligne ajoutée dans copie() :

```
copie.tourBlanc = this.tourBlanc;
```

Ce bug affectait aussi Minimax et AlphaBeta qui utilisent copie() via successeurs(). La correction a bénéficié à l'ensemble du projet.

Gestion du joueur courant dans les simulations

Deuxième problème : il ne faut pas alterner manuellement les joueurs pendant une simulation. Chaque successeur met déjà à jour le tour. Le joueur courant doit être lu directement depuis l'état :

```
Joueur joueurCourant = copie.getTourBlanc() ? Joueur.joueurBlanc :
Joueur.joueurNoir;
```

Tests : TestEvaluationMonteCarlo

Un test de diagnostic a simulé 20 parties depuis l'état initial avec une limite de 150 coups. Résultat : toutes les 20 parties avaient un gagnant, en 43 à 93 coups. Les simulations fonctionnent correctement.

En revanche, un test sur des feuilles à profondeur 2 a révélé une variance très élevée : sur la même feuille, cinq appels successifs retournaient des scores allant de -6 à +13 avec 100 simulations. C'est trop bruité pour que l'AlphaBeta prenne des décisions fiables.

13. Comparaison automatique des évaluations

ComparateurIA fait jouer deux évaluations l'une contre l'autre en deux parties (rôles inversés). Pour qu'A soit meilleure que B, A doit gagner les deux parties. La méthode comparer retourne 2 (A gagne), 1 (égalité), ou 0 (B gagne).

Booster contre Naive

Profondeur	Résultat	Interprétation
1	1/2 - égalité	Un coup ne suffit pas à exploiter le bonus

Profondeur	Résultat	Interprétation
		d'avancement
2	2/2 - Booster gagne	Le bonus positionnel fait la différence
3	2/2 - Booster gagne	Avantage confirmé avec plus de recul
2 (rôles inversés)	2/2 - Booster gagne	Indépendant de la couleur jouée

Booster > Naive à partir de la profondeur 2. À profondeur 1, les deux évaluations sont équivalentes car un seul coup ne suffit pas à exploiter l'information positionnelle.

Monte-Carlo contre Naive et Booster

Comparaison	Paramètres	Résultat
Monte-Carlo vs Naive	100 simulations, limiteCoups=80	0/2 - Naive gagne
Monte-Carlo vs Booster	100 simulations, limiteCoups=80	0/2 - Booster gagne

Monte-Carlo perd contre les deux autres. Les parties de Breakthrough durent ~70 coups en moyenne, ce qui rend chaque simulation coûteuse. À profondeur 2, l'AlphaBeta appelle eval sur ~400 feuilles, soit 40 000 parties simulées par coup. Avec 100 simulations, p reste trop bruité pour discriminer des positions proches. Pour obtenir une estimation fiable, il faut un grand nombre de simulations, ce qui devient incompatible avec Alpha-Beta où eval est appelé un grand nombre de fois.

Hiérarchie finale

Résultat
<i>Booster > Naive > Monte-Carlo dans le contexte de ce projet</i>

Monte-Carlo est mieux adapté à des jeux avec des parties courtes, ou utilisé sans AlphaBeta. Une heuristique simple mais bien choisie comme Booster surpasse Monte-Carlo quand le temps de calcul est contraint.

14. Menu principal

Un menu interactif en console permet de choisir le mode de jeu, la profondeur et l'évaluation. En mode IA/IA, l'évaluation de chaque joueur est configurée séparément. Les saisies invalides sont rejetées avec un message explicite.

15. Problèmes rencontrés et corrections

- Cases null : des vérifications explicites ont été ajoutées avant toute comparaison de cases.
- Génération des successeurs : confusion ligne/colonne dans le calcul de la case d'arrivée. Corrigé en séparant clairement directionVerticale (dépend du joueur) et direction (décalage de colonne).
- Fin de partie en IA/IA : le programme affichait AUCUN quand un joueur n'avait plus de coups. Corrigé en considérant que le joueur bloqué perd.
- Monte-Carlo -- copie() : tourBlanc non copié. Bug le plus critique, corrigé par une seule ligne (voir section 12).
- Monte-Carlo -- joueur courant : alternance manuelle remplacée par la lecture de copie.getTourBlanc().

16. Conclusion

Ce projet m'a permis de comprendre concrètement comment une IA de jeu se construit. L'algorithme Minimax n'est que la partie visible : tout repose sur la qualité des états, des successeurs et surtout de la fonction d'évaluation.

Les tests ont confirmé que Booster est la meilleure des trois évaluations dans ce contexte. Monte-Carlo, bien que fidèle à la méthode du cours, se heurte à la longueur des parties de Breakthrough. Une heuristique simple et bien pensée reste plus efficace que des simulations aléatoires coûteuses.

Des améliorations restent possibles : une interface graphique, une évaluation prenant en compte les menaces et les positions bloquées, ou un Monte-Carlo optimisé avec des simulations guidées plutôt que purement aléatoires.